

Computational Experiments with Stein Factorization in Macaulay2

An experimental technical note

July 12, 2026

Abstract

We implement a prototype of a Stein factorization algorithm in Macaulay2 and test it on several explicit projective maps. The implementation follows the bigraded Hom-module construction in Yasuda's algorithm:

$$C = \mathrm{Hom}_R(R_{\geq r}, R)_{(0, \geq 0)}.$$

The module is used to reconstruct a graded coordinate algebra for the Stein intermediate scheme and, where possible, the graph of the connected-fiber morphism. This note is not intended as a new theorem or a formal verification of the algorithm. It records a reproducible computational experiment, including selected Macaulay2 input and output. The code was developed with the assistance of an AI system; the mathematical setup, examples, expected structures, and checks were selected and reviewed by the author.

1 Purpose and background

For a projective morphism

$$f : Y \longrightarrow X,$$

a Stein factorization is a decomposition

$$Y \xrightarrow{h} Z \xrightarrow{g} X$$

in which h has connected fibers and g is finite. The purpose of this project is to turn an algorithm described by Takehiko Yasuda[1] into an executable Macaulay2 prototype and to see how it behaves on concrete examples from projective geometry.

The input is the graph of f represented as a bigraded projective scheme. If

$$R = S/I_{\Gamma_f},$$

then the variables in the source block have degree $(1, 0)$ and those in the target block have degree $(0, 1)$. The two gradings make it possible to separate the source direction from the target direction during the Hom-module computation.

2 The computational construction

For a sufficiently large bidegree $r = (r_1, r_2)$, the implementation computes

$$C = \mathrm{Hom}_R(R_{\geq r}, R)_{(0, \geq 0)}.$$

The truncation bound is obtained from shifts in a bounded bigraded free resolution of R over the ambient polynomial ring. This is the computational counterpart of the bound in Corollary 4.3 of [1].

The main public functions are:

```
data = steinHomData(S, graphIdeal, d1, d2, c1, c2)
cData = steinCoordinateAlgebra(data, gammaIndex, baseImages)
gData = directSteinGraph(data, cData)
```

The first function computes the truncated Hom module and records the bound. The second evaluates selected Hom generators at a nonzero element γ and computes the defining relations of the resulting coordinate algebra. The third computes a graph closure from the evaluated generators in a localization. For examples where the minimal resolution is too expensive, the experimental entry point `steinHomDataAtBound` accepts a supplied bound. Such a computation is explicitly marked as non-certified in the output.

3 Environment and reproducibility

The computations were run with Macaulay2 1.24.11 on macOS. The relevant bundled packages include `Truncations`, `MinimalPrimes`, and `Saturation`. From the project root, the standard suite is run by:

```
./run-tests.sh
```

The slower twisted-cubic benchmark is run separately:

```
M2 --no-readline --stop -q tests/blowup-twisted-cubic.m2
```

The complete input files are included in the repository. The excerpts below are chosen to show the mathematical input and the checks, without reproducing every elimination polynomial or the full Macaulay2 startup log.

4 Representative Macaulay2 calculations

4.1 A quadratic map of $\mathbb{P}^1$

The first test is the finite map $[s : t] \mapsto [s^2 : t^2]$. Its graph is described by $y_0^2 x_1 - y_1^2 x_0$. The following excerpt shows the input and the main output.

```
i = ideal(y0^2*x1-y1^2*x0);
d = steinHomData(S,i,1,1,2,2);
c = steinCoordinateAlgebra(d,0,{x0,x1});

d#"bound"
o = {1, 0}

d#"steinGeneratorDegrees"
o = {{0, 0}, {0, 1}}

c#"definingIdeal"
o = ideal (X0*X1-z^2)

isPrime (directSteinGraph(d,c)#"graphIdeal")
o = true
```

Thus the Stein intermediate is the source $\mathbb{P}^1$, displayed as the conic $X_0 X_1 - z^2 = 0$ over the target coordinate ring.

4.2 A Mori fiber space with a finite part

The next example is the composition

$$\mathbb{P}^1 \times \mathbb{P}^2 \longrightarrow \mathbb{P}^2 \xrightarrow{[x_0:x_1:x_2] \mapsto [x_0^2:x_1^2:x_2^2]} \mathbb{P}^2.$$

The source projection has connected $\mathbb{P}^1$ -fibers, while the second map is finite. The relevant part of the test input and output is:

```
igraph = kernel graphParametrization;
d = steinHomData(sgraph,igraph,5,2,6,3);
c = steinCoordinateAlgebra(d,0,{y0,y1,y2});

numgens igraph, codim igraph
o = (12, 4)

d#"bound"
o = {2, 0}

dim(c#"ring")
o = 3

hilbertFunction(1,c#"ring"), hilbertFunction(2,c#"ring")
o = (6, 15)

isPrime(c#"definingIdeal")
o = true

fiberComponents = minimalPrimes ifiber;
#fiberComponents
o = 4
```

Each of the four fiber components is a copy of $\mathbb{P}^1$. The computed Stein intermediate is the degree-two Veronese model of $\mathbb{P}^2$, and the connected part is the projection to $\mathbb{P}^2$.

4.3 The blow-up of a line in $\mathbb{P}^3$

We next take the blow-down $\text{Bl}_L(\mathbb{P}^3) \rightarrow \mathbb{P}^3$ along a line L , followed by the coordinatewise square map on $\mathbb{P}^3$. The main numerical checks are:

```
numgens igraph, codim igraph
o = (26, 7)

d = steinHomData(sgraph,igraph,7,3,8,4);
d#"bound", #d#"steinGeneratorIndices"
o = ({2, 0}, 8)

dim(c#"ring")
o = 4

hilbertFunction(1,c#"ring"), hilbertFunction(2,c#"ring")
o = (10, 35)
```

```

isPrime c#"definingIdeal"
o = true

degree(ssource/ie)
o = 2

#fiberPrimes, all(fiberPrimes,p -> degree(rfiber/p) == 1)
o = (8, true)

```

The Stein ring has the Hilbert values of the second Veronese ring of $\mathbb{P}^3$. The exceptional divisor has degree two and is the expected Segre quadric $\mathbb{P}^1 \times \mathbb{P}^1$. A representative fiber consists of eight reduced point components.

4.4 The twisted-cubic blow-up

The final example is the blow-up of $\mathbb{P}^3$ along the twisted cubic, followed by the coordinatewise square map. It is deliberately retained as a heavier benchmark. The full minimal resolution needed for the automatic Corollary 4.3 bound is expensive, so the test supplies $r = (2, 0)$ and records that this bound is experimental.

```

reesI = kernel map(targetRees,trees,{...});
numgens reesI, codim reesI
o = (3, 2)

igraph = saturate(igraphRaw,ideal matrix{flatten rows});
numgens igrph, codim igrph, isPrime igrph
o = (70, 11, true)

d = steinHomDataAtBound(sgraph,igrph,{2,0});
d#"steinGeneratorDegrees"
o = {{0,0},{0,1},{0,1},{0,1},{0,1},{0,1},{0,1},{0,2}}

hilbertFunction(1,c#"ring"), hilbertFunction(2,c#"ring")
o = (10, 35)

d#"certifiedBound"
o = false

```

The resulting ring again has the Hilbert values of the second Veronese of $\mathbb{P}^3$. One degree-two module generator is algebraically redundant, so automatic graph construction is not yet completed for this example. This is a useful illustration of the difference between a successful numerical stabilization and a certified bound.

5 Summary of results

The star indicates a supplied experimental bound rather than a bound computed from the full minimal resolution. Across these examples, the implementation reproduces the expected intermediate objects for finite maps, positive-dimensional connected fibers, and a divisorial contraction. The graph ideals produced by the direct localization strategy are prime in the completed examples.

Table 1: Main computational results

Example	bound	$\dim C$	selected Hilbert data	graph
$\mathbb{P}^1$ quadratic map	$(1, 0)$	2	$H(1) = 3$	prime
$\mathbb{P}^1$ cubic map	$(2, 0)$	2	twisted cubic	prime
$\mathbb{P}^1 \times \mathbb{P}^2$	$(2, 0)$	3	$H(1) = 6, H(2) = 15$	prime
$\text{Bl}_L(\mathbb{P}^3)$	$(2, 0)$	4	$H(1) = 10, H(2) = 35$	prime
Twisted-cubic blow-up	$(2, 0)^*$	4	$H(1) = 10, H(2) = 35$	partial

6 Limitations

This is a research prototype rather than a general-purpose Macaulay2 package. The main limitations are:

- deep minimal free resolutions can dominate the exact-bound computation;
- finite module generators may be redundant as algebra generators;
- heuristic stabilization gives evidence but does not prove the explicit bound;
- the component-selection part of the full algorithm is not automated for every possible input.

The calculations should therefore be read as reproducible computational evidence, not as a replacement for the mathematical hypotheses or proofs in the source paper.

7 Use of AI in the implementation

An AI system was used to help draft and revise Macaulay2 code, test cases, and explanatory documentation. The mathematical formulation, choice of examples, expected geometric interpretations, and verification criteria were supplied and reviewed by the author. The generated code was executed in Macaulay2, and the outputs were checked using defining ideals, Hilbert functions, primality tests, fiber decompositions, and composition checks. In particular, computations marked as non-certified were not presented as proofs.

8 Conclusion

This project shows that the bigraded Hom-module construction can be used in a practical Macaulay2 workflow on several nontrivial examples. The most useful outcome is not a large collection of examples, but a transparent record linking the mathematical construction, the source code, the actual computer input, and the numerical output. The repository is therefore intended as a public technical note and reproducible experiment rather than as a claim of a fully general implementation.

References

- [1] Takehiko Yasuda, *An algorithm for the minimal model program in dimension three*, arXiv:2603.13703v2. <https://arxiv.org/abs/2603.13703>
- [2] Daniel R. Grayson and Michael E. Stillman, *Macaulay2, a software system for research in algebraic geometry*. <https://math.uiuc.edu/Macaulay2/>